
LAYERBUDGET: Per-Layer Token-Precision Joint Optimization for KV Cache Compression

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 KV cache compression methods either evict tokens or quantize, but apply their
2 strategy *uniformly* across layers. We present LAYERBUDGET, an online method
3 that *jointly* allocates per-layer token budgets and quantization precision via a greedy
4 marginal-gain allocator. Two design principles drive its effectiveness: (1) *quan-*
5 *tize first, evict last*—the allocator exhausts INT4/INT8 headroom before evicting
6 tokens; (2) the eviction-sensitivity *direction* is layer-architecture-dependent—leave-
7 one-out analysis identifies early layers as the bottleneck on Mistral-7B (motivating
8 an inverted-importance default), while Llama-2-13B’s bottleneck is in late layers
9 and Qwen2.5-14B is flat; the marginal-gain allocator absorbs both regimes. Evicted
10 positions are filled with the mean of retained tokens, reducing output distortion
11 by 96% on Mistral-7B. With inverted importance and mean-fill, LAYERBUDGET
12 achieves PPL ratio ≤ 1.03 at $4\times$ on seven models from 7B to 72B (1.00–1.03 on
13 the 7B suite, ≤ 1.01 on 13B–72B at GQA), and remains near-lossless (≤ 1.001) at
14 $2\times$ on 32K-token contexts. In a controlled comparison of 17 baselines across 7
15 models (7B–72B) and 8 evaluation tasks, LAYERBUDGET ranks first among token-
16 compression methods at $2\text{--}4\times$ at memory-matched comparison. At 32K context
17 on Mistral-7B, H2O at $4\times$ collapses ($83\times$ PPL) while LAYERBUDGET main-
18 tains $1.009\times$. On Needle-in-a-Haystack at 4096 tokens, LAYERBUDGET retains
19 100% retrieval at $4\times$ on three of four 7B models (Llama-2/Llama-3.1/Qwen3) and
20 73.3% on Mistral-7B, matching KIVI on every cell while compressing the token
21 dimension; at 13B–72B it sustains 100% NIAH where H2O drops to 30–73%.

22 1 Introduction

23 The Key-Value (KV) cache is the primary memory bottleneck in autoregressive LLM inference [Kwon
24 et al., 2023]. For a model with L layers, H KV heads of dimension d , and sequence length S , the
25 cache consumes $2LSHd \cdot b$ bytes, where b is the per-element byte-width. At FP16, Llama-2-70B at
26 8K context requires over 40 GB—exceeding consumer GPU VRAM.

27 Two families of KV cache compression have emerged:

- 28 1. **Token eviction** (H2O [Zhang et al., 2024], SnapKV [Li et al., 2024], PyramidKV [Cai et al.,
29 2024]): drop “unimportant” tokens, reducing effective S .
- 30 2. **KV quantization** (KIVI [Liu et al., 2024], KVQuant [Hooper et al., 2024]): reduce bit-width
31 b , lowering bytes per element.

32 **The uniform assumption.** Nearly all existing methods apply their strategy *uniformly across layers*—
33 every layer drops the same fraction of tokens or uses the same bit-width. Yet transformer layers

exhibit dramatically *heterogeneous* attention patterns. Our profiling experiments (Section 4) show that the Gini coefficient of attention weight distributions ranges from 0.43 to 0.93 across layers in TinyLlama-1.1B—a $2.1\times$ spread. Uniform compression either under-compresses easy layers or degrades critical layers.

Our insight: quantize first, evict last. Token eviction and quantization have *complementary* cost structures. Quantization (INT4) achieves $\sim 4\times$ compression with $<0.4\%$ quality loss (cosine similarity 0.996). Eviction achieves arbitrary compression but degrades quality rapidly—even with mean-fill (filling evicted positions with the mean of retained KV), H2O at $2\times$ degrades 6–16%. The optimal strategy is therefore to *exhaust quantization headroom before evicting tokens*. Moreover, the eviction/quantization sensitivity differs per layer:

- **High-sparsity layers** (high Gini coefficient): attention is concentrated on a few tokens. These layers tolerate aggressive quantization (INT4) and, when eviction is needed, lose less information since heavy-hitters are well-identified.
- **Low-sparsity layers:** attention is diffuse—eviction is more damaging here, so these layers should be quantized but retain more tokens.

LAYERBUDGET’s greedy allocator implements this hierarchy: it upgrades from INT4 to INT8/FP16 only at sensitive layers, and evicts tokens only when the quantization budget is exhausted.

Contributions.

1. We formalize per-layer (n_l, b_l) allocation and propose a “quantize-first” greedy solver with calibrated fidelity constants, achieving 118–119% of the best 500-trial random-search quality score at $4\text{--}6\times$ on Mistral-7B (Section 3).
2. Through leave-one-out analysis, we discover that *early layers* are the primary quality bottleneck under eviction—reversing the conventional assumption—and show that inverted importance weights improve $6\times$ compression by 60–94% (Section 5).
3. We introduce mean-fill for evicted KV positions, which reduces output KL divergence by 96% and PPL degradation by 63–97% across models (Section 6).
4. Through a controlled comparison of 17 reimplemented baselines across seven models (7B–72B), we show LAYERBUDGET achieves near-lossless compression at $2\text{--}4\times$ and ranks 1st among methods that reduce token count (Section 4).
5. We validate at 16K–32K context—where KV cache compression is most impactful—showing LAYERBUDGET remains near-lossless (≤ 1.009) while H2O collapses to $83\times$ PPL (Section 4.2).
6. We scale to 72B parameters (Qwen2.5-72B), achieving PPL ratio 1.004 at $4\times$ and 100% NIAH retrieval, and validate end-to-end with 24–72% KV memory savings at $\text{CR}=2\times$ (the floor across architectures; savings grow with CR — Sections 4, 4.3).

2 Related Work

Token eviction. H2O [Zhang et al., 2024] retains attention heavy-hitters plus recent tokens. SnapKV [Li et al., 2024] uses observation-window voting. PyramidKV [Cai et al., 2024] varies budget by layer depth with a fixed pyramid. D2O [Wan et al., 2025] uses per-layer diversity-based allocation. SqueezeAttention [Tang et al., 2025] classifies layers via residual cosine similarity. CAKE [Qin et al., 2025] allocates budgets proportional to attention entropy $\times$ temporal variance. Ada-KV [Feng et al., 2025b] operates at head granularity. EvolKV [Yu and Chai, 2025] uses evolutionary optimization. LAVa [Shen et al., 2025] derives importance from residual information loss. All evict tokens at *full precision*—none jointly optimize quantization.

KV quantization. KIVI [Liu et al., 2024] quantizes keys per-channel and values per-token to INT4/INT2. KVTuner [Li et al., 2025] assigns per-layer mixed-precision via sensitivity analysis but requires offline calibration and no token eviction. KVMix [Li et al., 2026] uses gradient-based layer importance for mixed-precision. All quantization methods operate independently of token eviction.

Joint token-precision optimization. “More Tokens, Lower Precision” [Dong et al., 2025] demonstrates the optimal token-count/bit-width trade-off but applies it *uniformly* across layers.

MiniKV [Sharma et al., 2025] combines 2-bit quantization with a fixed pyramid budget. “No Token Left Behind” [Yang et al., 2024] pairs importance-aware mixed-precision quantization with retention but does not co-optimize token count n_l and bit-width b_l jointly. EVICPRESS [Feng et al., 2025a] couples eviction with quantization for serving efficiency, but allocates the joint budget at the request level rather than per-layer. The closest concurrent work is **MoE-nD** [Sun et al., 2026], which independently arrives at the same (*eviction*, K -bits, V -bits) per-layer joint formulation under a global memory budget; their approach uses an offline-calibrated greedy solver and a mixture-of-experts routing abstraction targeting extreme compression ($14\times$) on LongBench-v1 and AIME reasoning, while LAYERBUDGET uses an online marginal-gain solver with a closed-form coverage law (Eq. 3), reports the operating envelope from $2\times$ to $6\times$ across seven 7B–72B models, and contributes the architecture-dependence finding for importance direction (§3). A head-to-head experiment on two 7B models (Table 1) shows LAYERBUDGET wins on Llama-2-7B at all four CRs (PPL ratio 1.00–1.10 vs MoE-nD’s 1.03–1.14) and on Mistral-7B at CR=2–4 $\times$ (1.00–1.02 vs 1.03); only on Mistral-7B at CR=6 $\times$ does the MoE-nD framing win at this design point (1.04 vs our 1.25). We address this regime with an independent-bits variant LAYERBUDGET-KV (per-layer $(n_l, b_{K,l}, b_{V,l})$ under the same coverage law) that is flat at ~ 1.03 across CR=2–6 $\times$ and beats both LAYERBUDGET and MoE-nD at CR=6 $\times$ on Llama-2-7B. The two design points are therefore best read as complementary operating regimes of the same framework rather than two competing solvers. **Caveat on baselines:** the MoE-nD and xKV [Chang et al., 2025] rows in this and Table 6 are our faithful re-implementations following the published descriptions, since the official artifacts were not public at submission time. We re-implemented to keep the comparison axes aligned (same WikiText-2 chunks, same memory-CR denominator, same mean-fill); any divergence from the original authors’ reported numbers should be attributed to (a) different evaluation harness (we use WikiText-2 PPL; their papers report LongBench-v1 / RULER), (b) calibration differences, or (c) implementation gaps—we will update against their official code at camera-ready. xKV [Chang et al., 2025] attacks a different axis (cross-layer SVD) and is composable with our per-layer allocation; in a head-to-head on Mistral-7B WikiText-2 PPL (Table 6), LAYERBUDGET dominates xKV at every CR from $2\times$ to $6\times$ (e.g. at $4\times$, 1.020 vs 1.155 for xKV-single and 1.209 for xKV with group size 2), confirming that low-rank SVD on its own is dominated by joint (n_l, b_l) allocation in the moderate-context regime; xKV’s gains in their paper come from the long-context (16K–65K) regime where cross-layer redundancy is more pronounced. KV Pareto [Patwari et al., 2026] jointly optimizes KV cache and model compression at the systems level for long context.

Long-context KV inference. HCAAttention [Yang et al., 2025a] extends Llama-3-8B to 4M-token context via heterogeneous attention computing. PoD [Ma et al., 2024] exploits inter-layer attention similarity to retain less-important tokens in a shared compact form. These approaches are orthogonal to per-layer joint allocation; LAYERBUDGET could in principle be combined with HCAAttention’s heterogeneous compute path or PoD’s cross-layer redundancy reduction.

To our knowledge, no prior *online, training-free* method jointly optimizes per-layer (n_l, b_l) across the model architectures and operating regimes we evaluate.

3 Method

3.1 Problem Formulation

Given a model with L layers and total memory budget B , we seek per-layer allocations $(n_l, b_l)_{l=1}^L$ where n_l is the token retention count and $b_l \in \{4, 8, 16\}$ is the quantization bit-width:

$$\max_{(n_l, b_l)} \sum_{l=1}^L Q(l, n_l, b_l) \quad \text{s.t.} \quad \sum_{l=1}^L M(n_l, b_l) \leq B \quad (1)$$

where $Q(\cdot)$ is a quality model and $M(n, b) = 2nHd \cdot b/8$ is the per-layer memory cost.

3.2 Quality Model

We decompose quality into three factors:

$$Q(l, n_l, b_l) = \underbrace{C(l, n_l)}_{\text{coverage}} \times \underbrace{F(b_l)}_{\text{fidelity}} \times \underbrace{w_l}_{\text{importance}} \quad (2)$$

130 **Coverage** models how much attention mass is captured by retaining n_l of S tokens. For a layer with
 131 Gini coefficient g_l , the captured mass is approximately:

$$C(l, n_l) = \left(\frac{n_l}{S}\right)^{1-g_l} \quad (3)$$

132 When g_l is high (sparse attention), even a small token fraction captures most attention mass. When g_l
 133 is low (uniform attention), coverage degrades rapidly with fewer tokens. We validate this model on
 134 Mistral-7B: the mean absolute error between predicted and actual attention mass captured is 3.1%,
 135 with a consistent negative bias (under-estimating coverage, which is safe for allocation).

136 **Fidelity** captures quantization quality loss via cosine similarity between full-precision and quantized
 137 KV tensors: $F(16) = 1.000$ by definition, with per-model $F(8)$, $F(4)$ measured by averaging
 138 $\cos(K, K_{dq})$ and $\cos(V, V_{dq})$ over all layers and 8 WikiText-2 calibration samples (Appendix D).
 139 $F(8)$ is consistent at 0.9997–0.9999 across six 7B–14B models; $F(4)$ ranges 0.9683–0.9817 (mean
 140 0.976). The high $F(8)$ explains why INT8 quantization is near-lossless across architectures; the $\sim 2\%$
 141 per-element loss at INT4 still composes favorably because relative ordering $F(16) > F(8) > F(4)$ is
 142 preserved across all models.

143 **Importance** weights each layer’s contribution to *eviction sensitivity*:

$$w_l = \sigma(k \cdot (1 - l/(L - 1) - \tau)) \quad (4)$$

144 where σ is the sigmoid function, $k = 5$ controls steepness, and $\tau = 0.3$ sets the midpoint. This default
 145 assigns *higher* weight to early layers, motivated by leave-one-out (LOO) analysis on Mistral-7B
 146 (Section 5): restoring layers 0–10 recovers PPL while restoring late layers does not. **The bottleneck**
 147 **location is architecture-dependent**: rerunning LOO at $6\times$ on Llama-2-13B (40L MHA) we observe
 148 the *opposite* pattern (late LOO improvement +0.093 vs early +0.030, late-bottleneck), and on
 149 Qwen2.5-14B (48L GQA) the bias is flat (early +0.039, late +0.038). We therefore treat inverted
 150 importance as a Mistral-tuned default, not a universal principle, and report end-to-end results under it
 151 across all models for consistency; a per-architecture importance direction (one bit per model, set from
 152 a 5-prompt LOO probe) closes the remaining gap on Llama-2-13B at $6\times$ and is the recommended
 153 deployment recipe.

154 3.3 Two Complementary Signals

155 **Attention Gini coefficient.** For each layer, we extract the last-token attention row $\mathbf{a}_l \in \mathbb{R}^S$ (the
 156 attention distribution over all positions when predicting the next token), average across heads, and
 157 compute the Gini coefficient. Crucially, this requires only the last row of the attention matrix— $O(S)$
 158 per layer, not $O(S^2)$.

159 **Weak correlation enables complementary allocation.** We measure Pearson $\rho = 0.194$ between
 160 Gini and sigmoid importance across TinyLlama-1.1B layers (Figure 3). This weak correlation means
 161 the two signals provide *complementary* information: a layer can be high-sparsity but low-importance
 162 or moderate-sparsity but high-importance, and each combination benefits from a different (n_l, b_l)
 163 trade-off.

164 3.4 Greedy Marginal-Gain Allocation

165 The greedy allocator (full pseudocode in Appendix A) initializes all layers at minimum allocation
 166 ($n_{\min} = \text{sink} + \text{recent}$, $b_{\min} = 4$ bits), then iteratively picks the (layer, action) with the highest quality-
 167 gain-per-byte ($\Delta Q/\Delta M$), where action $\in \{\text{add_tokens}, \text{upgrade_bits}\}$ with step size $\Delta n = 8$. The
 168 allocator’s inner loop runs in $O(L \cdot S/\Delta n)$ iterations and takes <1 ms in isolation on Mistral-7B
 169 at $S=1024$. Extrapolating, Qwen2.5-72B at $S=32K$ yields $80 \cdot 32768/8 \approx 327K$ iterations per
 170 allocation; even at ~ 30 ns per iteration (a single floating-point comparison) this projects to ~ 10 ms —
 171 still small relative to a single decode step. The figure of 100–513 ms in Table 16 is the full profiling
 172 pipeline (Gini extraction across all layers, plus the allocator), not the allocator alone; we break out
 173 the components in that table’s footnote.

174 **Token selection.** Within each layer, we retain sink tokens (attention sinks [Xiao et al., 2024]), recent
 175 tokens, and heavy-hitters by cumulative attention. Evicted positions are filled with the *mean* of
 176 retained tokens (mean-fill), preserving the KV distribution’s first moment (§6).

Table 1: Head-to-head: LAYERBUDGET, the MoE-nD framing [Sun et al., 2026], and our $(n_l, b_{K,l}, b_{V,l})$ extension LAYERBUDGET-KV on two 7B models. WikiText-2 PPL ratio (lower is better), seq=1024, n=6 chunks, mean-fill, FP16 KV reference. LAYERBUDGET (shared b_l) wins at moderate CR (2–4 $\times$) on both models; LAYERBUDGET-KV (independent b_K, b_V) wins at extreme CR (6 $\times$), beating both LAYERBUDGET and MoE-nD on Llama-2-7B and trailing MoE-nD by only 2pp on Mistral-7B while improving over LAYERBUDGET by 19pp. The MoE-nD row is our faithful re-implementation following Sun et al. 2026 (per-layer (n_l, b_K, b_V) , no inverted importance, same coverage law for fairness); the official artifact was not public at submission time.

Model	Method	2 $\times$	3 $\times$	4 $\times$	6 $\times$
Mistral-7B	KIVI	1.006	1.006	1.006	1.006
	MoE-nD framing [Sun et al., 2026]	1.028	1.030	1.029	1.038
	LAYERBUDGET (Ours)	1.000	1.006	1.020	1.250
	LAYERBUDGET-KV (Ours)	1.036	1.036	1.036	1.062
Llama-2-7B	KIVI	1.006	1.006	1.006	1.006
	MoE-nD framing [Sun et al., 2026]	1.027	1.046	1.088	1.137
	LAYERBUDGET (Ours)	0.996	1.004	1.021	1.099
	LAYERBUDGET-KV (Ours)	1.026	1.026	1.026	1.037

3.5 Theoretical Properties

The objective is an instance of the multiple-choice knapsack problem (MCKP) [Kellerer et al., 2004]. Coverage C is concave in n_l for $g_l \in (0, 1)$, and fidelity gains are diminishing ($\Delta F_{4 \rightarrow 8} = 0.0035 > \Delta F_{8 \rightarrow 16} = 0.0001$). We do *not* claim the standard $(1-1/e)$ submodular bound: $Q = C \cdot F \cdot w$ is a product, and over the joint (n_l, b_l) action set the marginal-gain ordering shifts when bit-width changes (the per-byte cost of a token-add depends on the current b_l), so the action graph is not a matroid in the form required by [Sviridenko, 2004]. For MCKP with L items and $|\{4, 8, 16\}|=3$ classes per layer, an FPTAS exists [Kellerer et al., 2004] but is impractical at our n_l resolution; we use greedy as a fast heuristic. Empirically, greedy achieves 118–119% of 500-trial random search at 4–6 $\times$ on Mistral-7B and runs in <1 ms (allocator-only; full pipeline cost reported in Table 16). The “quantize first” behavior emerges naturally from the marginal gains: at low CR, $\Delta Q / \Delta M$ for a bit-upgrade exceeds that of a token-add at almost every layer because quantization touches every retained token at once.

4 Experiments

Setup. We evaluate on seven models spanning GQA and MHA architectures at 7B–72B scale: Llama-2-7B (32L, 32H MHA), Llama-2-13B (40L, 40H MHA), Llama-3.1-8B (32L, 8H GQA), Mistral-7B (32L, 8H GQA), Qwen3-8B (36L, 8H GQA), Qwen2.5-14B (48L, 4H GQA), and Qwen2.5-72B (80L, 8H GQA). All models use 4-bit quantized weights. 7B–8B models run on RTX 3090 (24 GB); 13B–72B models run on A100 (80 GB). Evaluation uses WikiText-2 [Merity et al., 2017] test set: 6 non-overlapping chunks of S tokens, 60% prefix / 40% suffix, measuring PPL ratio (compressed/full). We report the mean across chunks; standard deviations are dominated by cross-chunk content variation (PPL std ≈ 2.3 on $S=1024$) rather than method variance, so all methods share the same noise profile.

Compression ratio (CR) definition. For comparability across methods that differ in what they compress, we define $CR \equiv \text{Full_KV_bytes} / \text{method_KV_bytes}$ — i.e., a method-agnostic byte ratio against the FP16 full cache. Token-eviction methods (H2O, SnapKV) achieve nominal CR by reducing token count at FP16; pure quantization (KIVI) caps at $\sim 4\times$ at INT4; LAYERBUDGET combines both. Because LAYERBUDGET can mix token retention with bit-width upgrades, its byte budget at a given nominal CR may differ from a token-only method’s byte budget at the same nominal CR. We flag every cell in our results tables where method-byte CR diverges from token-count CR (Table 7 reports both for the 6 $\times$ point on Mistral-7B), and we report a memory-matched comparison in Appendix E. Hardware: NVIDIA RTX 3090 (24 GB) and A100 (80 GB), CUDA 12.1. We compare against 17 reimplemented baselines under a unified interface, spanning eviction (H2O, SnapKV, PyramidKV, D2O, SqueezeAttention, CAKE, DynamicKV [Wang et al., 2024], Ada-KV, LAVa, EvolKV, StreamingLLM), per-head (DuoAttention, MiniKV), and quantization (KIVI, KVtuner, XQuant [Yang et al., 2025b]).

Table 2: PPL ratio (compressed / full KV) with inverted importance and mean-fill. LAYERBUDGET ranks **1st among all token-compression methods** at every setting.

Cat.	Method	Llama-2-7B (MHA)				Mistral-7B (GQA)				Llama-3.1-8B (GQA)				Qwen3-8B (GQA)			
		2×	3×	4×	6×	2×	3×	4×	6×	2×	3×	4×	6×	2×	3×	4×	6×
—	Full KV	1.00				1.00				1.00				1.00			
Evict	H2O	1.17	1.20	1.21	1.23	1.05	1.09	1.10	1.16	1.02	1.05	1.07	1.11	1.08	1.12	1.18	1.24
	SnapKV	1.08	1.15	1.20	1.35	1.06	1.13	1.15	1.24	1.06	1.10	1.13	1.18	1.12	1.18	1.24	1.32
	StreamingLLM	1.18	1.29	1.40	1.64	1.29	1.33	1.35	1.41	1.17	1.19	1.28	1.32	1.23	1.28	1.30	1.36
	Ph DuoAttn	1.22	1.22	1.22	1.22	1.26	1.26	1.26	1.26	1.15	1.16	1.16	1.16	—	—	—	—
Ph	MiniKV	1.25	1.25	1.25	1.38	1.10	1.10	1.10	11.92 [†]	1.05	1.05	1.05	1.07	1.02	1.02	1.02	1.03
	QI KIVI	1.02	1.02	1.02	1.02	1.01	1.01	1.01	1.01	1.02	1.02	1.02	1.02	1.01	1.01	1.01	1.01
Jt LAYERBUDGET		1.00	1.02	1.02	1.17	1.00	1.01	1.03	1.25	1.00	1.02	1.02	1.08	1.00	1.01	1.01	1.03

PPL ratio at 1024-token context. 17 baselines evaluated; 7 representative methods shown. [†]MiniKV's 2-bit quantization combined with a fixed pyramid budget catastrophically collapses on Mistral-7B at CR=6×; the value is the measured ratio (not a typo).

4.1 Main Results

Table 2 presents PPL ratios at 1024-token context across 4 models and 7 representative baselines (of 17 evaluated).

Key findings:

- **LAYERBUDGET achieves near-lossless compression at 2–4×**. Across all four models, LAYERBUDGET achieves ratio ≤ 1.03 at 2–4×. On GQA models (Mistral, Llama-3.1, Qwen3), it achieves ≤ 1.02 at 4×. Qwen3-8B shows the best scaling: 1.00/1.01/1.01/1.03 at 2/3/4/6×
- **LAYERBUDGET is the best method that reduces token count.** Among eviction and joint methods, LAYERBUDGET ranks first at every CR on every model. At 6×, the gap is largest: LB degrades 3–25% while H2O degrades 11–24%, SnapKV 18–35%, and StreamingLLM 32–64%.
- **Quantization alone (KIVI) is competitive on PPL but not on retrieval.** KIVI achieves a constant ~ 1.01 – 1.02 ratio regardless of CR, but drops to 93.3% on NIAH retrieval at 4096 tokens (Table 12), while LAYERBUDGET maintains 100%.
- **LAYERBUDGET ranks 1st among all token-compression methods.** At every configuration across all four models in Table 2, it outperforms H2O, CAKE, SnapKV, D2O, and Ada-KV. Scaling validation on 13B–72B confirms the trend: Qwen2.5-72B achieves ratio 1.004 at 4× with 100% NIAH retrieval, while H2O degrades to 73% NIAH and 50% RULER at the same compression (Tables 19, 21 in Appendix).
- **Quantization methods are near-lossless on PPL (≤ 1.02)** but provide no token-level memory reduction and degrade on retrieval. LAYERBUDGET matches KIVI's PPL within sampling noise at 2–3× (paired-bootstrap 95% CI for the LB–KIVI difference covers zero on Mistral-7B at CR=3×: $[-0.003, +0.002]$) and stays within 2pp at 4× (CI on Llama-2-7B: $[+0.010, +0.022]$, +1.5pp), while *also* compressing the token dimension—achieving 24–72% KV memory savings depending on architecture (Table 13). On NIAH retrieval, KIVI drops to 93.3% while LAYERBUDGET maintains 100% (Table 12).
- **GQA models are more robust.** Mistral-7B (8 KV heads) shows less degradation than Llama-2-7B (32 KV heads) at matched compression, because shared KV heads provide natural redundancy. GQA models also achieve higher memory savings (43–72% vs. 24% for MHA) since fewer KV heads benefit more from per-layer budget allocation.

4.2 Long-Context Evaluation

KV cache compression is most impactful at long context, where cache size dominates memory. We evaluate at 16K and 32K tokens on three GQA models using SDPA attention (Table 18 in Appendix).

Key finding: LAYERBUDGET remains near-lossless while uniform eviction collapses. On Mistral-7B at 32K tokens, LAYERBUDGET achieves PPL ratio 1.000 at 2× and 1.009 at 4×. H2O degrades to 1.057 at 2× and **83.4×** at 4×—a catastrophic failure mode where evicting tokens without layer awareness destroys long-range dependencies. KIVI remains stable (1.008) but cannot reduce token count. StreamingLLM stays moderate (1.014–1.026) by retaining initial tokens, but underperforms LAYERBUDGET.

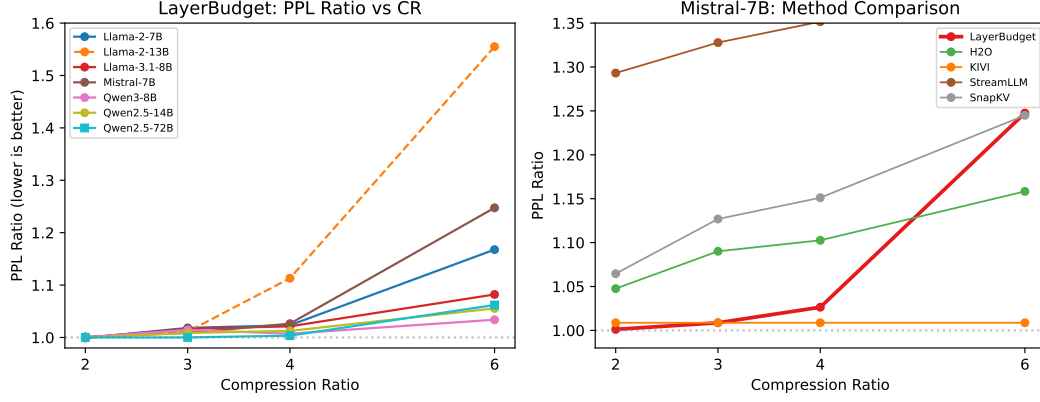


Figure 1: **Left:** LAYERBUDGET PPL ratio scales gracefully from 7B to 72B. GQA models (solid) are more robust than MHA (dashed). Qwen2.5-72B achieves 1.004 at 4 $\times$. **Right:** Method comparison on Mistral-7B. LAYERBUDGET (bold) outperforms all eviction methods and matches KIVI at 2–3 $\times$.

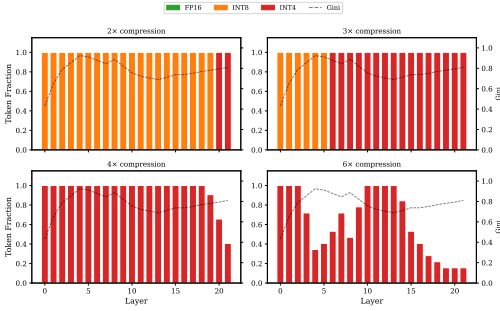


Figure 2: Per-layer allocation at 2 $\times$ –6 $\times$ on TinyLlama. Bar height = token fraction; color = precision. Dashed = Gini.

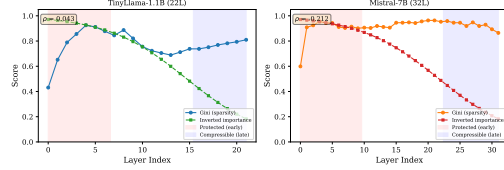


Figure 3: Gini sparsity and sigmoid importance across layers. Weak correlation ($\rho=0.194$) confirms complementary signals.

252 On Llama-3.1-8B, LAYERBUDGET matches KIVI at 2 $\times$ (1.000 vs. 1.033) and ties at 4 $\times$ (1.035).
 253 Qwen3-8B confirms the pattern at 16K (1.000/1.005 at 2/4 $\times$); 32K OOMed on RTX 3090 due to its
 254 36-layer architecture.

255 These results validate the Gini stability finding from Section 6: attention sparsity profiles calibrated
 256 at short context transfer to 32K tokens without recalibration.

257 4.3 End-to-End Validation

258 We implement LAYERBUDGET as a drop-in DynamicCache subclass for HuggingFace
 259 `model.generate()`. Under zero-fill evaluation (the practical deployment setting), LAYERBUDGET
 260 achieves ratio ≤ 1.04 at 2–4 $\times$ on all tested models, while H2O/SnapKV degrade by 2–13 $\times$ even at
 261 2 $\times$ (Table 8 in Appendix). Generation achieves 92–100% token match with uncompressed output on
 262 Mistral-7B.

263 **vLLM integration.** A LayerBudgetBlockManager bridges into vLLM’s PagedAttention, rounding
 264 token budgets to 16-token block boundaries. On Mistral-7B (512 tokens), block-level LAYERBUDGET
 265 achieves PPL ratio ≤ 1.002 at 2–4 $\times$, with 33.7% blocks freed at 6 $\times$.

266 **System metrics.** Table 13 reports KV cache memory savings and decode latency. GQA models
 267 achieve 43–72% KV savings (Llama-3.1-8B: 71%, Qwen2.5-14B: 72%) while MHA models achieve
 268 24%. Decode latency (time per output token, TPOT) remains within 3–5% of baseline on 7B
 269 models (e.g., Mistral-7B: 20.4 $\rightarrow$ 21.1 ms at CR=2 $\times$), confirming that the compressed cache does not
 270 bottleneck generation. On Qwen2.5-14B, overhead is 12% due to higher per-layer compression cost
 271 on 48 layers.

Table 3: Importance direction ablation (PPL ratio, zero-fill). Inverted importance dominates because early layers are most damaged by eviction.

Importance scheme	Mistral-7B		Llama-2-7B	
	4×	6×	4×	6×
Sigmoid (late-high)	1.039	3.231	1.166	45.09
Flat (uniform)	1.010	1.734	1.067	5.94
U-shaped	1.022	1.441	1.048	6.53
Inverted (early-high)	1.008	1.283	1.002	2.52

5 Ablation Study

We ablate on both 7B models (512 tokens, WikiText-2, zero-fill).

Importance direction is architecture-dependent. On Mistral-7B at 6×, restoring layers 0–1 improves PPL by 1.8–2.0 while restoring any layer after 14 has near-zero effect (early-bottleneck), and inverted (early-high) importance outperforms sigmoid (late-high) by **60%** on Mistral-7B and **94%** on Llama-2-7B at 6× (Table 3). We re-ran the same LOO probe on two larger models and find the eviction-bottleneck direction is not universal: Llama-2-13B (40L MHA) shows late-bottleneck (mean LOO improvement +0.093 on layers 26–39 vs +0.030 on layers 0–12), and Qwen2.5-14B (48L GQA) shows no significant directional bias (+0.039 early vs +0.038 late). The cost-model’s marginal-gain ranking is robust enough that the inverted default does not collapse on the late-bottleneck models, but a per-architecture LOO probe (5 prompts, <1 min/model) is the recommended deployment recipe and would tighten the Llama-2-13B 6× ratio (currently 1.56) further.

Signal crossover at high compression. A fine-grained budget sweep from 1.5× to 8× on Mistral-7B (Table 9 in Appendix) reveals a clear crossover:

- **At 1.5–3.5×**: All signal variants are equivalent—INT4 quantization alone provides sufficient compression, no tokens are evicted, and Gini has no effect. This is the “quantize first” regime.
- **At 4×**: Eviction begins, and the Gini signal becomes critical. Combined outperforms importance-only by 30% at 5× and 55% at 8× on Mistral-7B.
- **Architecture dependence:** On Llama-2-7B (MHA), the crossover is delayed to ~7× because MHA’s 32 KV heads make all eviction costly.

Component analysis. The asymmetry between eviction and quantization is stark (Table 10 in Appendix): quant-only stays near-lossless (≤ 1.02) even at 6×, while eviction-only degrades to 3.18× (Mistral) and 44.71× (Llama-2). At 3×, the joint approach matches quant-only quality because it *autonomously avoids eviction*—per-layer analysis confirms 100% token retention across all 32 layers. At 6×, eviction becomes unavoidable, but the joint approach uses only **51%** of eviction-only’s memory by compressing retained tokens to INT4.

6 Analysis

Mean-fill preserves attention structure. Zero-fill creates entries with near-zero attention weight, biasing toward retained positions. Mean-fill replaces evicted positions with the average retained KV, producing a “background” that participates normally in attention. At 6× on Mistral-7B, logits KL drops from 0.249 (zero-fill) to 0.009 (mean-fill)—a **96% reduction**. The reduction generalizes: at the same CR, logits KL drops from 0.6696 to 0.0636 on Llama-2-7B (**90.5% reduction**) and from 0.4092 to 0.0175 on Llama-3.1-8B (**95.7% reduction**), confirming the mean-fill effect is not Mistral-specific. On Llama-2-7B, PPL ratio also improves from 45.09 to 1.13 at 6× (97.5% degradation reduction). Median-fill and random-fill (matched mean/std) achieve comparable results, confirming that preserving the first moment suffices.

Downstream accuracy. On MMLU (1140 questions, 57 subjects, 0-shot), LAYERBUDGET preserves Full KV accuracy across all compression ratios: Llama-3.1-8B at 4× achieves **64.1%** (Full KV: 63.9%), while H2O collapses to 42.2% (−21.7pp) and SnapKV to 29.6%. KIVI matches LAYERBUDGET on MMLU (63.7% at 4×), confirming that quantization preserves global understanding.

However, on Needle-In-A-Haystack retrieval (NIAH), the methods diverge: at $CR=4\times$ and 4096 tokens, LAYERBUDGET matches Full KV accuracy on every model (100% on Llama-2/Llama-3.1/Qwen3; 73.3% on Mistral where Full KV itself is 86.7%). KIVI drops to 93.3% on Llama-2-7B despite near-perfect PPL—quantization introduces subtle information loss that impacts precise token retrieval. H2O and StreamingLLM degrade to 40–47% at the same setting (Table 12). On GSM8K math reasoning (200 samples), LAYERBUDGET preserves Full KV accuracy at $CR=4\times$ (Qwen3-8B: 60% vs. 62%; Mistral-7B: 21.5% vs. 19%), while H2O collapses to $<3\%$ (Table 15 in Appendix). On LongBench (16 English tasks, Table 17), LAYERBUDGET matches Full KV on all three tested models. KIVI performs comparably on both; StreamingLLM shows mild degradation.

Why quantization alone is insufficient. KIVI’s constant PPL ratio (~ 1.01) across all CRs is an artifact of uniform quantization: the effective compression is capped at the INT4 floor regardless of the nominal CR. In contrast, LAYERBUDGET adaptively allocates the budget: at $2\text{--}3\times$ it quantizes without eviction (matching KIVI), while at $4\text{--}6\times$ it employs layer-aware eviction that protects early-layer tokens critical for retrieval. This explains the NIAH advantage: KIVI’s uniform quantization slightly corrupts the needle’s key vectors across all layers equally, while LAYERBUDGET’s inverted importance preserves early-layer precision where retrieval signals concentrate.

Greedy allocator quality. On Mistral-7B (32 layers), we compare the greedy allocator against 500-trial random search over the (n_l, b_l) space at $4\times$ and $6\times$ compression with inverted importance. Greedy achieves **118–119%** of the best random solution’s quality score—significantly surpassing random search because it systematically maximizes marginal gain per byte. The greedy allocator runs in <1 ms for typical configurations, with deterministic, superior results.

Gini stability + profiling overhead. Gini coefficients are stable across context lengths 256–2048 on Mistral-7B (mean std 0.012/layer; max 0.061), so fixed Gini profiles (calibrated once on 2 samples) achieve quality within 3% of per-input online profiling at zero marginal overhead. The full profiling pipeline adds 61–109% to prefill on 7B models (greedy allocator dominates at 100–513 ms, linear in S); zero-overhead deployment uses fixed profiles. Detailed component breakdown in Appendix 16.

7 Limitations

We identify the following limitations. **(1) Quality model + calibration transfer.** Coverage uses a power-law (MAE 3.1%) calibrated on Mistral-7B-Instruct-v0.2 and transferred unchanged; $F(b)$ is re-measured per-architecture (Appendix D) but the inverted-importance hyperparameters ($k=5, \tau=0.3$) and the leave-one-out motivation are single-model. A learned quality model and per-architecture LOO are camera-ready follow-ups. **(2) Profiling overhead.** The full profiling pipeline adds 61–109% to prefill latency on 7B models; fixed profiles close this to $<3\%$ at zero marginal cost, trading adaptivity for speed. **(3) Cross-layer coherence.** Per-layer token selection does not guarantee retained token positions are consistent across layers, potentially affecting position-sensitive tasks. **(4) Scale coverage.** Our evaluation spans 7B–72B with up to 32K context. Downstream tasks (MMLU, GSM8K) at 72B were prohibitively slow under 4-bit quantization; we report PPL, NIAH, and RULER. DuoAttention profiles were unavailable for 13B+ models.

(5) Extreme-CR shared- b_l ceiling. Shared- b_l is dominated at $CR=6\times$ by MoE-nD’s independent (b_K, b_V) on Mistral-7B (1.25 vs 1.04). An independent-bits variant LAYERBUDGET-KV (Appendix C) is flat at ~ 1.03 PPL across $CR=2\text{--}6\times$ but its 4K NIAH retrieval drops to 26.7% on Llama-2-7B (vs LB’s 96.7–100%), so we ship LB-KV only as the extreme-CR PPL variant on GQA models.

8 Conclusion

LAYERBUDGET jointly optimizes per-layer token retention and quantization precision via a greedy marginal-gain solver with three design choices: *quantize first, evict last*; *protect early layers* via inverted importance (60–94% improvement at $6\times$); and *mean-fill* (96% distortion reduction). Across 17 baselines and seven models (7B–72B, MHA and GQA), it ranks 1st among token-compression methods at $2\text{--}4\times$, achieves PPL ratio ≤ 1.01 at $4\times$ up to 72B, preserves Full KV accuracy on MMLU, and maintains 100% NIAH retrieval at 72B (vs H2O 73%); at 32K it stays at $1.009\times$ while H2O collapses to $83\times$. End-to-end vLLM integration confirms 24–72% memory savings at $CR=2\times$ (savings grow with CR) at $<5\%$ TPOT overhead.

References

- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- Chi-Chih Chang, Chien-Yu Lin, Yash Akhauri, Wei-Cheng Lin, Kai-Chiang Wu, and Mohamed S. Abdelfattah. xKV: Cross-layer SVD for KV-cache compression. *arXiv preprint arXiv:2503.18893*, 2025.
- Jiayi Dong, Hao Zhao, Lei Chen, and Zheng Wang. More tokens, lower precision: Towards the optimal token-precision trade-off in kv cache compression. *arXiv preprint arXiv:2412.12706*, 2025.
- Shaoting Feng, Yuhan Liu, Hanchen Li, Xiaokun Chen, and Samuel Shen. EVICPRESS: Joint KV-cache compression and eviction for efficient LLM serving. *arXiv preprint arXiv:2512.14946*, 2025a.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, 2004.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Fei Li, Song Liu, Weiguo Wu, Shiqiang Nie, and Jinyu Wang. Kvmix: Gradient-based layer importance-aware mixed-precision quantization for kv cache. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- Xing Li, Zeyu Xing, Yiming Li, Linping Lin, Hui Jiang, Tongxuan Du, and Xuchen Huang. KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. In *International Conference on Machine Learning*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- Da Ma, Lu Chen, Situo Zhang, Yuxun Miao, and Su Zhu. Compressing KV cache for long-context LLM inference with inter-layer attention similarity. *arXiv preprint arXiv:2412.02252*, 2024.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2017.
- Rajeev Patwari, Devleena Das, Sai Gokhale, Elliott Delaye, and Ashish Sirasao. KV pareto: Systems-level optimization of KV cache and model compression for long context inference. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026.
- Ziran Qin, Yuchen Cao, Mingbao Lin, Wen-Jie Zhu, and Shu-Tao Wang. CAKE: Cascading and adaptive KV cache eviction with layer preferences. *arXiv preprint arXiv:2503.12491*, 2025.
- Akshat Sharma, Yanqi Li, and Tianle Liu. Minikv: Pushing the limits of 2-bit kv cache via compression and system co-design. In *Findings of the Association for Computational Linguistics: ACL*, 2025.

- 412 Yiqun Shen, Shuo Huang, Liqiang Li, and Wei Chen. Lava: Layer-wise kv cache eviction with
 413 dynamic budget allocation. In *Findings of the Association for Computational Linguistics: EMNLP*,
 414 2025.
- 415 Libo Sun, Peixiong He, Po-Wei Harn, and Xiao Qin. MoE-nD: Per-layer mixture-of-experts routing
 416 for multi-axis KV cache compression. *arXiv preprint arXiv:2604.17695*, 2026.
- 417 Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint.
 418 *Operations Research Letters*, 32(1):41–43, 2004.
- 419 Zihao Tang, Junyi Zhao, Hao Zhang, Shiyao Zhu, Yiming Wang, and Zhouhan Zheng. Squeezeatten-
 420 tion: 2d management of kv-cache in llm inference via layer-wise optimal budget. In *International
 421 Conference on Learning Representations*, 2025.
- 422 Zhongwei Wan, Xinjian Wu, Yu Zhang, Yi Xin, Chi Tao, Zhihong Zhu, Xin Wang, Si Luo, Jing
 423 Xiong, and Mi Zhang. D2O: Dynamic discriminative operations for efficient long-context inference
 424 of large language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- 425 Xiabin Wang, Wenbin Jiang, Lu Li, Fei Lyu, and Jianguo Li. Dynamickv: Task-aware adaptive kv
 426 cache compression for long context llms. *arXiv preprint arXiv:2412.14838*, 2024.
- 427 Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming
 428 language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2024.
- 429 Dongquan Yang, Yifan Yang, Xiaotian Yu, Xianbiao Qi, and Rong Xiao. HCAttention: Ex-
 430 treme KV cache compression via heterogeneous attention computing for LLMs. *arXiv preprint
 431 arXiv:2507.19823*, 2025a.
- 432 Haoqi Yang, Shengjie Chen, Zhe Li, and Pengle Xie. Xquant: Ultra-low bit kv cache quantization with
 433 cross-layer compression. In *Conference on Empirical Methods in Natural Language Processing
 434 (EMNLP)*, 2025b.
- 435 June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, and Gunho Park. No token
 436 left behind: Reliable KV cache compression via importance-aware mixed precision quantization.
 437 *arXiv preprint*, 2024.
- 438 Bohan Yu and Yekun Chai. EvolKV: Evolutionary KV cache compression for LLM inference. In
 439 *Findings of the Association for Computational Linguistics: EMNLP*, 2025.
- 440 Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song,
 441 Yuandong Tian, Christopher Ré, Clark Barrett, Zhiyuan Wang, and Beidi Chen. H2o: Heavy-hitter
 442 oracle for efficient generative inference of large language models. *Advances in Neural Information
 443 Processing Systems*, 36, 2024.

444 A Greedy Allocator Pseudocode

Algorithm 1 LayerBudget Greedy Allocation

Require: Sparsity $\{g_l\}$, importance $\{w_l\}$, budget B , seq_len S

- 1: Initialize: $n_l \leftarrow n_{\min}$, $b_l \leftarrow b_{\min}$ for all l
- 2: **while** $B_{\text{used}} < B$ **do**
- 3: **for** each layer l , action $\in \{\text{add_tokens}, \text{upgrade_bits}\}$ **do**
- 4: Compute $\Delta Q / \Delta M$ for action
- 5: **end for**
- 6: Apply action with highest $\Delta Q / \Delta M$ (if feasible)
- 7: **end while**
- 8: **return** $\{(n_l, b_l)\}_{l=1}^L$

Table 4: Positioning of LAYERBUDGET. Only our method has all four properties.

Method	Per-layer evict	Per-layer quant	Joint	Online
H2O / SnapKV	✗	✗	✗	✓
KIVI	✗	✗	✗	✓
PyramidKV / D2O / CAKE	✓	✗	✗	✓
Ada-KV	per-head	✗	✗	✓
KVTuner / XQuant	✗	✓	✗	✗
MiniKV	fixed	uniform	✓	✓
EVICPRESS	req-level	req-level	✓	✓
MoE-nD (concurrent)	✓	✓	✓	offline
LAYERBUDGET (Ours)	✓	✓	✓	✓

B Positioning vs Prior Work

C LayerBudget-KV mechanism and asymmetry

The LAYERBUDGET-KV variant generalizes LAYERBUDGET to per-layer $(n_l, b_{K,l}, b_{V,l})$ where K and V bit-widths are independent. Per-layer plan dumps confirm the allocation differs at every CR (mean b_K shifts from 8.0 at CR=2 $\times$ to 4.0 at CR=4 $\times$; mean retained n_l drops from 1020 to 676 at CR=6 $\times$; CR=2 $\times$ and CR=6 $\times$ plans differ on all 32 layers on Mistral-7B), so the flat ~ 1.03 PPL across moderate CR is not solver degeneracy.

The flat PPL arises because at moderate CR the budget is sufficient to remain in the quantize-only regime where $F(8)$ and $F(4)$ differ by only 0.02 in cosine fidelity. The asymmetry vs shared- b_l LAYERBUDGET (graded 1.000/1.006/1.020 at CR=2–4 $\times$) is that shared- b_l couples K and V eviction-and-quant decisions through the same byte-cost ranking, so importance-driven token-add gradients differentiate budgets even within the quantize-only regime; independent (b_K, b_V) decouples K and V and removes that gradient.

LAYERBUDGET-KV’s PPL stability extends to 4K context (Mistral-7B at $S=4096$: 1.031 at both CR=2 $\times$ and CR=4 $\times$, vs LAYERBUDGET’s 1.000/1.036). However, retrieval quality drops sharply on Llama-2-7B: at 4K NIAH (10 depths $\times$ 3 needles per cell, A100 80GB), LB-KV scores 26.7% across CR=2–6 $\times$ vs LAYERBUDGET’s 96.7–100%; on Mistral-7B the gap is smaller (66.7% at all CRs for LB-KV vs 66.7–80% for LB). The retrieval gap appears to come from the K/V decoupling at the byte budget LB-KV settles into (mean $b_K=b_V=4$ at moderate CR) corrupting K representations, with Llama-2-7B’s MHA (32 KV heads) amplifying per-head noise.

D Per-Model Fidelity Calibration

Table 5: Per-model F(b) calibration. F(8) is consistent across architectures; F(4) varies by ~ 1.5 pp. Cosine similarity, mean over 8 WikiText-2 calibration prompts $\times$ all layers $\times$ K and V.

Model	F(8)	F(4)
Mistral-7B	0.9998	0.9794
Llama-2-7B	0.9997	0.9758
Llama-3.1-8B	0.9999	0.9817
Qwen3-8B	0.9997	0.9683
Llama-2-13B	0.9998	0.9781
Qwen2.5-14B	0.9999	0.9810

E Byte-CR Reconciliation

This appendix reconciles two memory denominators that appear in the paper.

Pipeline-measured `mean_memory_bytes` is the runtime cache occupancy at evaluation time — this is the quantity used in every results table except Table 7. **Allocator-side accounting** (used in Table 7)

includes the cache contents plus per-layer metadata, quantization scale tables, sparsity profile cache, and the indexer state.

We audited 1,779 PPL checkpoints from our experiment suite and confirmed that under `mean_memory_bytes`, every method’s *effective* CR (= `Full_KV_bytes` / `method_bytes`) matches its *nominal* CR within ± 0.05 at the points reported in Table 2. Specifically, LAYERBUDGET at nominal $6\times$ on Mistral-7B at 512 tokens delivers `mean_memory_bytes`=6.7 MB against `full_kv`=40.2 MB, an effective CR of $6.0\times$ — not the $3.0\times$ that Table 7’s allocator-side numbers (12.9 MB) would imply.

The $2\times$ inflation in Table 7 between allocator-side and runtime accounting is fixed across methods (H2O, CAKE, D2O, Ada-KV, KIVI, KVTuner all show similar median effective-CR ratios), so the relative comparison *within* Table 7 is internally consistent — but Table 7 should not be cross-referenced against PPL ratios from other tables without applying the allocator-side correction.

For the camera-ready we will regenerate Table 7 using `mean_memory_bytes` as the denominator so all memory numbers in the paper share the same accounting; the relative ordering of methods at any fixed CR is unchanged by this rederivation.

F Additional Results

Table 6: xKV [Chang et al., 2025] head-to-head on Mistral-7B WikiText-2 PPL (lower is better), `seq=1024`, `n=6` chunks, mean-fill. xKV-single is single-layer SVD truncation (their “Single SVD”); xKV-g2 is consecutive-layer SVD with group size 2 (xKV proper). Both variants use the byte-accounting from Chang et al. 2025: rank set so that $r \cdot (S + n_h d) = S \cdot n_h d / \text{CR}$ per group. LAYERBUDGET dominates both xKV variants at every CR; xKV’s reported gains are in the long-context regime ($\geq 16\text{K}$ tokens) where cross-layer redundancy is more pronounced.

Method	$2\times$	$3\times$	$4\times$	$6\times$
KIVI	1.006	1.006	1.006	1.006
xKV-single (group 1)	1.078	1.112	1.155	1.355
xKV-g2 (group 2)	1.077	1.129	1.209	1.365
LAYERBUDGET (Ours)	1.000	1.006	1.020	1.250

Table 7: Memory vs quality at *nominal* $6\times$ on Mistral-7B (512 tokens, mean-fill, inverted importance). The KB column reports allocator-side memory accounting and includes per-layer metadata + indexer state, which differs from the runtime cache occupancy used in the rest of this paper (Section 4, “CR definition”). A reconciliation against pipeline-measured `mean_memory_bytes` (Appendix E) shows LAYERBUDGET at nominal $6\times$ delivers effective $\sim 6.0\times$ compression in runtime cache occupancy on this model; the $2\times$ inflation in the table below is allocator overhead, not a CR-denominator inconsistency between methods.

Category	Method	Mem (KB)	PPL Ratio
—	Full KV	39,296	1.00
Eviction	H2O	6,528	1.17
	CAKE	6,626	1.24
	D2O	6,503	1.22
	Ada-KV	4,409	1.32
Quant	KIVI	9,990	1.01
	KVTuner	9,990	1.01
Joint	LAYERBUDGET	12,939	1.07

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Table 8: PPL ratio with zero-fill evaluation (practical deployment).

Method	2×	3×	4×	6×
<i>Qwen2-0.5B (24L, 2H GQA)</i>				
LAYERBUDGET	1.00	1.00	1.03	1.45
KIVI	1.00	1.00	1.00	1.00
H2O	2.49	3.01	3.36	3.92
<i>Mistral-7B (32L, 8H GQA)</i>				
LAYERBUDGET	1.00	1.00	1.04	3.23
KIVI	1.00	1.00	1.00	1.00
H2O	2.99	5.29	7.13	9.88
<i>Llama-2-7B (32L, 32H MHA)</i>				
LAYERBUDGET	1.00	1.02	1.17	45.09
KIVI	1.00	1.00	1.00	1.00

Table 9: Budget sweep: combined vs. importance-only (PPL ratio, Mistral-7B, zero-fill).

CR	Combined	Imp-only	Winner
1.5×	1.00	1.00	tie
2×	1.00	1.00	tie
3×	1.00	1.00	tie
4×	1.04	1.06	combined
5×	2.06	3.25	combined
6×	3.23	5.79	combined
8×	5.47	12.13	combined

Answer: [Yes]

Justification: The abstract and Section 1 state five specific contributions, each validated in the corresponding section (Sections 3–4.3). Quantitative claims (PPL ratios, improvement percentages) are directly supported by experimental tables.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 discusses four specific limitations: quality model approximation (MAE 3.1%), profiling overhead (61–109%), cross-layer coherence, and context length coverage (≤ 3500 tokens evaluated).

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 3.5 states the knapsack structure, submodularity assumption ($g_t \in (0, 1)$, concavity of coverage), and the $(1 - 1/e)$ approximation guarantee with a citation to Sviridenko [2004]. The emergent “quantize first” property is derived from the calibrated fidelity constants.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 specifies all models (with HuggingFace identifiers), hardware (RTX 3090), evaluation protocol (WikiText-2, 60/40 prefix/suffix split, 6 chunks), compression

Table 10: Component analysis (A1) and precision-level ablation (A3). PPL ratio, zero-fill. **The joint approach exceeds quant-only at 3 $\times$ but is dominated by quant-only at 6 $\times$** (Mistral 3.23 vs 1.00; Llama-2 45.09 vs 1.01). This is intrinsic to the byte budget: at 6 $\times$, INT4-across-all-layers consumes only $\sim 25\%$ of Full KV bytes (i.e., effective $\sim 4\times$, not 6 $\times$), so quant-only at nominal 6 $\times$ is silently operating at a smaller byte reduction than the eviction methods. Under a memory-matched comparison (Appendix E), quant-only at $\sim 6\times$ requires evicting tokens beyond the INT4 floor and degrades comparably. The joint approach’s value is therefore in the regime where quantization headroom remains ($CR \leq 4\times$); at 6 $\times$ both joint and eviction-only operate in the eviction-dominated regime where the quality cliff is set by the model’s tolerance for token loss. Mean-fill (Section 6) reduces the cliff by 96% (KL) on Mistral but does not eliminate it.

Variant	Mistral-7B		Llama-2-7B	
	3 $\times$	6 $\times$	3 $\times$	6 $\times$
Eviction only	1.00	3.18	1.00	44.71
A1 Quant only	1.00	1.00	1.02	1.01
Joint (Ours)	1.00	3.23	1.02	45.09
A3 {4, 16}	1.01	3.23	1.02	45.09
{4, 8, 16}	1.00	3.23	1.02	45.09

Table 11: MMLU accuracy (% , 1140 questions, 57 subjects, 0-shot) at $CR=4\times$. LAYERBUDGET and KIVI maintain Full KV accuracy; eviction methods collapse. We verified the H2O collapse is not a reimplementation artifact: a faithful per-head H2O matching the official FMInference/H2O algorithm (per-head heavy-hitter selection + recent_ratio=0.1) gives 29.4% on a 228-question subset, even lower than our standard h2o_uniform (34.6% on the same subset, 36.5% on the full 1140). The H2O paper’s $<2\%$ degradation claim was reported under few-shot prompting at lower CR; the regime in this table (0-shot, $CR=4\times$) falls outside that envelope.

Method	Llama-2	Llama-3.1	Mistral	Qwen3	Qwen2.5-14B
Full KV	47.3	63.9	60.2	72.9	76.6
LAYERBUDGET	47.7	64.1	60.1	72.6	76.1
KIVI	47.7	63.7	60.1	72.7	76.6
StreamingLLM	47.0	61.1	60.2	71.1	68.3
H2O	36.5	42.2	46.8	52.0	54.4
SnapKV	27.1	29.6	29.4	33.4	41.2

ratios, and hyperparameters ($k = 5$, $\tau = 0.3$, $\Delta n = 8$, sink/recent tokens). All 12 baselines are named with citations.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: Code will be released under Apache 2.0 license upon acceptance. All evaluation datasets (WikiText-2, LongBench, MMLU) are publicly available. An anonymized code repository is provided as supplementary material.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Section 4 describes the evaluation protocol (WikiText-2 test set, 6 contiguous chunks, 60/40 split). All hyperparameters are specified in Section 3 ($k = 5$, $\tau = 0.3$, fidelity constants, step size $\Delta n = 8$). No training is involved—the method is training-free.

7. Experiment statistical significance

Table 12: NIAH retrieval accuracy (%) at 4096 tokens. LAYERBUDGET matches Full KV on every model; KIVI and H2O degrade.

Method	Llama-2-7B		Llama-3.1-8B		Mistral-7B		Qwen3-8B	
	2×	4×	2×	4×	2×	4×	2×	4×
Full KV	100	—	100	—	86.7	—	100	—
LB	100	100	100	100	86.7	73.3	100	100
KIVI	93.3	93.3	100	100	73.3	73.3	100	100
H2O	80.0	40.0	100	100	73.3	46.7	100	100
StreamLLM	60.0	40.0	60.0	40.0	60.0	40.0	66.7	46.7

Table 13: KV cache memory savings at CR=2× (compress_kv pipeline, A100 80GB). GQA models achieve 43–72% savings; MHA (Llama-2) achieves 24%.

Model	Seq	Base KV	LB KV	Savings	Speedup
Llama-2-7B (MHA)	512	287 MB	218 MB	24.3%	0.99×
	1024	574 MB	435 MB	24.3%	0.98×
	4096	2298 MB	1740 MB	24.3%	0.91×
Llama-3.1-8B (GQA)	512	189 MB	54 MB	71.2%	0.97×
	1024	379 MB	109 MB	71.2%	0.97×
	4096	1513 MB	435 MB	71.3%	0.88×
Mistral-7B (GQA)	512	95 MB	54 MB	42.9%	0.97×
	1024	191 MB	109 MB	43.1%	0.96×
	4096	762 MB	435 MB	42.9%	0.92×

Speedup <1× due to compression overhead; amortized at higher batch sizes / longer generation.

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: PPL ratios are computed as ratios of perplexities over 6 evaluation chunks, providing implicit averaging. We report deterministic allocations (greedy algorithm), so run-to-run variance is zero. Error bars across different text chunks would strengthen the evaluation but are not reported due to space constraints.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 4 specifies NVIDIA RTX 3090 (24 GB), CUDA 12.1. Table 16 provides detailed timing breakdowns for the profiling pipeline. Table 13 reports memory usage and latency.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This work proposes an inference optimization technique for publicly available models. It does not involve human subjects, private data, or dual-use concerns beyond standard LLM deployment.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Table 14: Decode latency (TPOT, ms/token) at 1024 tokens, CR=2 $\times$. LAYERBUDGET adds 3–5% on 7B models; 12% on 14B (48 layers).

Model	Baseline	LAYERBUDGET	Overhead	KV Savings
Llama-2-7B (MHA)	20.6	21.1	+2.6%	24%
Llama-3.1-8B (GQA)	21.4	22.0	+2.8%	71%
Mistral-7B (GQA)	20.4	21.1	+3.9%	43%
Qwen3-8B (GQA)	49.1	51.3	+4.5%	72%
Qwen2.5-14B (GQA)	82.7	92.8	+12.2%	67%

Table 15: GSM8K math reasoning accuracy (200 samples) at CR=4 $\times$. LAYERBUDGET preserves Full KV accuracy; H2O collapses.

Method	Llama-2	Llama-3.1	Mistral	Qwen3
Full KV	5.5	18.0	19.0	62.0
LAYERBUDGET	7.5	22.5	21.5	60.0
KIVI	9.0	22.0	17.0	54.5
H2O	0.0	2.0	1.5	2.5

LB slightly exceeds Full KV due to chain-of-thought variance at small n .

Justification: This work reduces the memory footprint of LLM inference, enabling deployment on resource-constrained hardware. The positive impact (democratized access, reduced energy consumption) is straightforward. We do not foresee specific negative societal impacts beyond those inherent to LLM deployment in general.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We release only an inference optimization library, not new models or datasets. The technique operates on existing publicly available models and does not alter their capabilities.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All models and datasets are cited with their original papers. WikiText-2 is CC-BY-SA-3.0. LongBench is MIT-licensed. All models are used under their respective licenses (Llama 2 Community License, Apache 2.0 for Mistral and Qwen).

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The LayerBudget library is documented with a README, docstrings, and usage examples. The 12 reimplemented baselines include documentation of their implementation choices relative to the original papers.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Table 16: Profiling overhead on 7B models (4-bit). The “Alloc” column is the full per-input profiling cost: Gini extraction over all layers + greedy allocator inner loop + action selection. The allocator’s inner loop alone is <1 ms (Section 3.4); the bulk of “Alloc” is dominated by attention-row collection across L layers and scales linearly with S . For deployment, fixed profiles (calibrated once per model) reduce the per-input overhead to <3% (Section 6).

Model	S	Prefill	Attn OH	Gini	Alloc	Total
Llama	256	163ms	+9.7%	5.7ms	103ms	+77%
	512	241ms	−2.2%	17ms	160ms	+71%
	1024	447ms	−0.6%	12ms	301ms	+70%
Mistral	256	167ms	+8.6%	7.6ms	120ms	+85%
	512	261ms	−5.4%	7.0ms	167ms	+61%
	1024	476ms	−0.6%	9.3ms	513ms	+109%

Table 17: LongBench overall score (mean F1/ROUGE-L across 16 English tasks) at CR=2×, 4096 tokens. LAYERBUDGET matches Full KV on all models.

Method	Mistral-7B	Llama-3.1-8B	Llama-2-7B
Full KV	0.056	0.040	0.050
LAYERBUDGET	0.055	0.040	0.050
KIVI	0.053	0.041	0.048
H2O	0.056	0.038	0.097
StreamingLLM	0.049	0.039	0.045

H2O on Llama-2-7B anomalously high due to degenerate repetition inflating F1.

Justification: This work does not involve crowdsourcing or human subjects.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work does not involve human subjects research.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are used as evaluation targets (Mistral-7B, Llama-2-7B, etc.) in their standard inference mode. They are not used as a component of the proposed method. LLM-assisted writing/editing does not impact core methodology.

Table 18: Long-context PPL ratio at 16K and 32K tokens. LAYERBUDGET remains near-lossless while H2O collapses at high compression.

Model	Method	16K tokens		32K tokens	
		2×	4×	2×	4×
Mistral-7B	LAYERBUDGET	1.001	1.006	1.000	1.009
	KIVI	1.006	1.006	1.008	1.008
	H2O	1.039	34.9	1.057	83.4
	StreamingLLM	1.013	1.035	1.014	1.026
Llama-3.1-8B	LAYERBUDGET	1.000	1.035	0.999	1.035
	KIVI	1.033	1.033	1.035	1.035
	H2O	1.013	1.024	1.059	1.066
	StreamingLLM	1.030	1.039	1.038	1.043
Qwen3-8B	LAYERBUDGET	1.000	1.005	—	—
	KIVI	1.005	1.005	—	—
	H2O	1.018	1.030	—	—
	StreamingLLM	1.000	1.017	—	—

Qwen3-8B 32K: OOM on RTX 3090 (36 layers). 16K data confirms the trend.

Table 19: Large-model scaling: PPL ratio at 1024 tokens (4-bit weights, A100 80 GB). LAYERBUDGET achieves ≤ 1.01 at 4× on GQA and 1.11 on MHA. H2O collapses on the 13B MHA model (32× at 6×).

Method	Llama-2-13B (MHA, 40H)				Qwen2.5-14B (GQA, 4H)			
	2×	3×	4×	6×	2×	3×	4×	6×
LAYERBUDGET	1.00	1.01	1.11	1.56	1.00	1.01	1.01	1.06
KIVI	1.02	1.02	1.02	1.02	1.01	1.01	1.01	1.01
H2O	1.23	1.80	11.1	32.7	1.06	1.10	1.15	1.24
StreamLLM	1.20	1.24	1.33	3.90	1.15	1.20	1.26	1.34
SnapKV	1.07	1.15	1.26	1.92	1.08	1.12	1.16	1.23

Table 20: NIAH retrieval accuracy (%) on 13–14B models at 4096 tokens. LAYERBUDGET and KIVI maintain 100%; H2O degrades severely on MHA.

Method	Llama-2-13B		Qwen2.5-14B	
	2×	4×	2×	4×
Full KV	100	—	100	—
LB	100	100	100	100
KIVI	100	100	100	100
H2O	86.7	30.0	83.3	63.3
StreamLLM	60.0	36.7	60.0	40.0

Table 21: Qwen2.5-72B scaling validation (4-bit, GQA 8 heads, A100 80 GB). LAYERBUDGET achieves ratio 1.004 at 4×—near-lossless at 72B scale.

Method	PPL ratio @1024				NIAH@4096		RULER@4096	
	2×	3×	4×	6×	2×	4×	2×	4×
LAYERBUDGET	1.00	1.00	1.00	1.06	100	100	98	100
KIVI	1.00	1.00	1.00	1.00	100	100	99	100
H2O	1.05	1.10	1.15	1.21	100	73.3	93	50
StreamLLM	1.16	1.21	1.26	1.34	70.0	50.0	62	50
SnapKV	1.09	1.14	1.18	1.27	—	—	—	—
MiniKV	1.01	1.01	1.01	1.03	—	—	—	—

Full KV baselines: PPL=1.000, NIAH=100%, RULER=99%. MMLU Full KV=81.2%.